

Important: Not Available in notes4.md

This file contains only the parts from `preparation.md` that were marked as not available, not directly available, or only inferred from `notes4.md`.

Question 14: MPI communication and mesh network

Not available in `notes4.md`: MPI is not directly explained.

MPI stands for Message Passing Interface. In the climate simulation scenario, processors use MPI to exchange data because each processor has its own local memory. Neighboring processors exchange boundary data after each computation step. At the end, all processors send partial results to a master processor.

Partially not available in `notes4.md`: mesh networks are not explained in detail.

A mesh interconnection network connects each processor to nearby neighboring processors. This works well for climate simulation because each processor mainly needs boundary data from adjacent regions.

Question 16: FinTech-specific Flynn's Taxonomy examples

Not available in `notes4.md`: FinTech-specific examples are not given.

- SISD: A simple program processing one transaction at a time.
- SIMD: Market-data vector calculations, risk scoring over many records, or GPU-based analytics.
- MISD: Theoretical redundant checking of the same transaction data through different validation methods.
- MIMD: Fraud detection, batch processing of trading history, payment processing, and predictive analytics.

Most modern FinTech systems commonly use MIMD because they depend on multicore CPUs, clusters, and distributed servers. SIMD accelerators may also be used for matrix-heavy analytics.

Question 17: Robotics-specific architecture recommendation

Not directly available in `notes4.md`: the exact robotics scenario is not given, so this recommendation is inferred from the notes.

- Central control unit: use a scalar or superscalar general-purpose processor because it handles decision-making, control flow, and coordination.
- Vision processing module: use a SIMD array processor or GPU-style architecture because images contain many pixels that can be processed with the same operation.
- High-speed numerical computation unit: use vector or array processors because matrix operations are data-parallel.
- Embedded real-time controllers: use RISC because it gives fast response, simpler instruction decoding, pipelining support, and low power consumption.
- If compatibility with older software or compact code is required, CISC can be considered, but it is less suitable for low-power real-time embedded control.

Question 18: Healthcare-specific Flynn's Taxonomy examples

Not available in `notes4.md`: healthcare-specific examples are not given.

- SISD: A standalone program processing one patient's record sequentially.
- SIMD: Medical image analysis, where the same image-processing operation is applied to many pixels.
- MISD: Theoretical fault-tolerant checking of the same patient signal by different algorithms.
- MIMD: Real-time monitoring, disease-outbreak prediction, hospital databases, and distributed analytics systems.

Most modern healthcare analytics systems commonly use MIMD because they often depend on multicore CPUs, clusters, and distributed systems. SIMD can also be used for specialized image-processing workloads.

Question 19: Attached array processor limitation

Partially not available in `notes4.md`: limitations of attached array processors are only lightly covered.

An attached array processor can be limited by host communication and I/O transfer overhead. Since it works as an auxiliary processor connected to a host computer, performance may depend on how quickly data can move between the host, the I/O interface, the attached processor, and local memory.

Question 22: Client-server architecture in a hospital system

Not available in `notes4.md`: client-server architecture is not explained.

In a hospital client-server system, the central server stores and manages patient data. Doctors, laboratories, and billing departments act as clients that send requests to the server.

This supports parallel usage because many clients can access the system at the same time. For example:

- A doctor can view patient history.
- A lab can upload test results.
- Billing can update payment records.
- The server coordinates access to shared patient data.

To keep the data correct, the server must use synchronization, access control, and transaction management so that simultaneous updates do not corrupt patient records.

Question 23: DNA sequence comparison example

Not directly available in `notes4.md`: DNA-specific examples are not given, so the example is inferred from the notes.

Parallel computing improves DNA sequence comparison by dividing the large comparison task into smaller parts and processing them at the same time on multiple processors.

For example, different processors can compare different sequence segments. SIMD or GPU-style processing can apply the same comparison operation across many data items. MIMD systems can also run different parts of the analysis on different datasets.

The final results can be combined after all processors finish their assigned work.

Question 24: Distributed system models for ride-sharing

Not available in `notes4.md`: peer-to-peer, master-slave, and hybrid distributed system models are not explained.

Peer-to-peer model:

- Nodes communicate directly with each other.
- Nearby drivers and users may exchange local updates through nearby services.
- It reduces dependence on one central point, but coordination and security are harder.

Master-slave model:

- A central master server coordinates the system.
- Worker servers process driver locations, ride requests, pricing, and route data.
- It is easier to control, but the master can become a bottleneck if not replicated.

Hybrid model:

- Combines central coordination with distributed regional servers.

- Regional servers handle local driver updates and ride matching.
- Central servers manage global data, payments, policies, and monitoring.

For Uber/Careem-like systems, a hybrid distributed model is usually best because it supports real-time regional processing while still keeping central control.